Experiment 5: GNU Debugger (GDB) - Program Flow Analysis

```
 * This program demonstrates key program flow concepts:
    - Function calls and returns
    - Loops
    - Conditional branching
    - Recursive functions
    - The call stack

  Compile with debug symbols:
   gcc -g -O0 exp5_program_flow.c -o exp5_program_flow

  Run with GDB:
    gdb ./exp5_program_flow


#include <stdio.h>

/* ------------------------------------------------------
 * Function: add
 * Adds two integers and returns the result.
 * Useful for observing how arguments are passed on the stack.
 * ------------------------------------------------------ */
int add(int a, int b) {
    int result = a + b;      /* Set breakpoint here to inspect 'result' */
    return result;
}

/* ------------------------------------------------------
 * Function: factorial (Recursive)
 * Computes n! recursively.
 * Useful for observing how the call stack grows with each
 * recursive call and shrinks as calls return.
 * ------------------------------------------------------ */
int factorial(int n) {
    if (n == 0 || n == 1) {   /* Base case */
        return 1;
    }
    return n * factorial(n - 1);  /* Recursive call */
}

/* ------------------------------------------------------
 * Function: find_max
 * Finds the maximum value in an integer array.
```

```
 * Demonstrates loop-based flow and conditional branching.
 * ------------------------------------------------------ */
int find_max(int arr[], int size) {
    int max = arr[0];

    for (int i = 1; i < size; i++) {    /* Loop: observe i and max changing */
        if (arr[i] > max) {             /* Branch: only updates when new max found */
            max = arr[i];
        }
    }
    return max;
}

/* ------------------------------------------------------
 * Function: is_even
 * Returns 1 if a number is even, 0 if odd.
 * Simple conditional to observe branch prediction.
 * ------------------------------------------------------ */
int is_even(int n) {
    if (n % 2 == 0) {
        return 1;
    } else {
        return 0;
    }
}

/* ------------------------------------------------------
 * main: Entry point
 * Calls each function in sequence to demonstrate program flow.
 * ------------------------------------------------------ */
int main() {
    printf("=== Experiment 5: Program Flow Analysis with GDB ===\n\n");

    /* --- 1. Simple function call --- */
    printf("--- Section 1: Function Call ---\n");
    int x = 10, y = 25;
    int sum = add(x, y);                /* Step INTO add() to see the stack frame */
    printf("add(%d, %d) = %d\n\n", x, y, sum);

    /* --- 2. Loop demonstration --- */
    printf("--- Section 2: Loop ---\n");
    int numbers[] = {3, 7, 1, 9, 4, 6, 2, 8, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int max = find_max(numbers, size);  /* Observe loop iterations inside find_max() */
```

```
    printf("Max value in array: %d\n\n", max);

    /* --- 3. Conditional branching --- */
    printf("--- Section 3: Conditional Branching ---\n");
    for (int i = 1; i <= 6; i++) {
        if (is_even(i)) {              /* Branch: alternates between even/odd */
            printf("%d is EVEN\n", i);
        } else {
            printf("%d is ODD\n", i);
        }
    }
    printf("\n");

    /* --- 4. Recursive function (call stack grows) --- */
    printf("--- Section 4: Recursion & Call Stack ---\n");
    for (int n = 1; n <= 6; n++) {
        int fact = factorial(n);       /* Use 'backtrace' in GDB here to see stack depth */
        printf("%d! = %d\n", n, fact);
    }
    printf("\n");

    printf("=== Program Completed ===\n");
    return 0;
}
```

OUTPUT:

```
PS C:\VIKRANTH\NIT Delhi\2nd Year - 2\COA> cd "c:\VIKRANTH\NIT Delhi\2nd Year - 2\COA\" ; if ($?) { gcc gnd.c -o gnd } ; if ($?) { .\gnd }
=== Experiment 5: Program Flow Analysis with GDB ===

--- Section 1: Function Call ---
add(10, 25) = 35

--- Section 2: Loop ---
Max value in array: 9

--- Section 3: Conditional Branching ---
1 is ODD
2 is EVEN
3 is ODD
4 is EVEN
5 is ODD
6 is EVEN

--- Section 4: Recursion & Call Stack ---
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720

=== Program Completed ===
```